黑龙江科技大学

本科毕业设计开题报告

题　　目：基于 C++高并发架构与语言大模型的交互系统的开发

学　　院：计算机与信息工程学院

班　　级：软件 22-1 班

姓　　名：周殿波

学　　号：2022024763

指导教师：高辉　　侯亚斌

教师职称：讲师　　工程师

黑龙江科技大学本科毕业设计开题报告

题　　目	基于C++高并发架构与语言大模型的交互系统的开发
题目来源	☐科研项目　☑生产和社会实践　☐教学(含实验)辅助　☐创新创业项目
成果类别	☑毕业设计　　☐毕业论文
是否在实验、实习、工程实践和社会调查等社会实践中完成：　　☑是　　☐否	

1、研究目的和意义（需说明设计产品的主要用途和应用领域）

随着人工智能技术的爆发，大语言模型展现出强大的自然语言处理能力，在人工智能领域产生了巨大影响[1]，LLM 已成为研究热点。在大规模数据上预训练之后，它具有强大的少样本和零样本上下文学习能力，能够便捷地用于许多真实场景复杂任务[2]。

在高流量的智能咨询服务、企业内部知识库助理以及在线教育互动平台、电商大促期间的智能客服等智能交互领域，语言大模型的接入能极大提升用户转换率，提高企业效益。

但将这些“大脑”高效地部署到生产环境，并支撑成千上万用户的同时访问，是当前互联网大厂面临的核心挑战。本研究目的为设计并实现一个能够支撑高并发请求的新型智能交互系统。

传统智能客服在大规模并发访问时易卡死、崩溃，且对语义的理解十分肤浅，问题回答生硬。本系统能提供基于大语言模型的自然语言理解与生成功能，对话流畅且回答自然准确。

在通信方面，实现底层高性能与上层灵活性的平衡。提供基于 Reactor 模式的高性能网络监听与处理功能，确保系统在高负荷下稳定运行。

2、国内外发展情况(文献综述)

（1）国内发展情况

当前国内互联网大厂（如腾讯、百度、字节跳动）广泛采用 Linux 环境下 C++ 作为底层核心，虽然 C++要求程序员自己管理对象的生命期，这在多线程环境下显得尤为困难，但以 C++为底层进行开发，随之对应的是显著高于其他语言的运行速度[3]。

采用基于 Linux 的高并发网络编程模型和 TinyWebServer 为代表的 Reactor 架构，以应对海量用户并发压力。随着国产大语言模型（如智谱 GLM、DeepSeek）的爆发式增长，国内开发者也开始探索将大模型与实际业务的结合。从简单的 API 调用转向研究 AI 智能体，以及研究如何构建稳

定的后台系统来承载智能推理业务。

（2）国外发展情况

国外在分布式系统架构与AI部署技术方面起步较早，技术体系非常成熟。在高性能服务器领域，国外对I/O多路复用（Epoll/Kqueue）以及事件驱动模型的研究极深。

在AI推理引擎方面，国外开源社区贡献了大量如FastAPI的Python异步框架，使AI模型的服务化部署（Model-as-a-Service）变得极其便捷。国际前沿聚焦于如何利用高性能编程语言对AI模型进行加速，即AI Infra。

（3）目前存在待解决的问题

虽然高性能服务器和大语言模型各自的发展都非常迅速，但在实际结合中仍存在许多待解决的问题。纯Python服务器（如Flask）虽然开发AI业务灵活，但在高并发场景下性能极差；而纯C++开发门槛极高且周期太长。

在高并发请求下，AI推理的巨大算力消耗往往会导致服务器响应超时，需要设计合理的线程池与定时器管理机制来维持系统稳定。且LLMs在处理复杂任务尤其是在任务需要结合规划及外部工具调用的场合时仍存在局限性[4]。

3、研究/设计的目标

本课题要求结合C++高并发架构与语言大模型，搭建一套可稳定承载多用户并发访问的智能交互系统。

系统要求具备高性能网络通信能力，支持基于套接字 API 的非阻塞通信，可高效处理用户的HTTP 请求，实现请求的快速接收、解析与转发；实现自然语言智能交互功能，依托大语言模型完成精准的用户意图解析与自然流畅的问答反馈，解决传统智能客服语义理解浅、回答生硬的问题；支持多角色权限管理：管理员可实现服务器资源监控、线程池参数动态调优、API 密钥权限分配；业务主管可完成 AI 提示词（Prompt）策略优化、行业垂直领域知识库关联与更新；普通用户可进行自然语言咨询、复杂业务逻辑查询、数据智能化汇总分析。

系统要求具备完善的连接管理与任务调度能力，可自动清理超时非活动连接，合理分配线程资源，避免系统资源泄漏与浪费，在高频突发流量场景下保持极低的响应延迟。AI 推理请求无明显超时，可稳定输出准确的推理结果；长时间高负载运行时，内存占用平稳，无明显资源泄漏问题。采用前后端分离，降低系统调试与维护复杂度，提升架构的可扩展性，探索一套“底层高性能、上层强智能”的可扩展交互架构，探索AI工程化部署的可行方案。

4、设计方案（研究/设计方法、理论分析、计算、实验方法和步骤等）

在系统架构方面，本系统使用采用解耦的双后端微服务架构，技术路线如下。

开发工具：VS code，Pycharm。

版本管理：Git。

开发环境：Linux 虚拟机。

核心技术：C++高性能开发,python AI 引擎。

数据库：MySQL。

前端接入层：采用 C++编写高性能 Web 服务器，作为流量的第一道防线，利用 Linux 中特有的 I/O 复用函数的 Epoll，将事件放在内核时间表中，无需重复传入文件描述符集或事件集,实现非阻塞通信[6]，确保能够支撑高并发的用户连接。

业务逻辑层：采用 Python 的 FastAPI 框架构建 AI 推理微服务，负责解析用户意图并调用前沿大语言模型 API 获取答案。

通信机制：两端通过本地 HTTP/POST 协议进行高效通信。C++服务器在接收请求后，作为“代理客户端”将数据转发给 Python 服务，并在获取结果后回传给用户。

在功能实现方面，系统主要包括以下模块：

1.网络通信模块(C++)：基于 Reactor 模式，利用 Epoll 监控套接字事件，解析为完成数据交换而约定的协议[7]，实现单线程处理万级并发连接的能力。

2.任务并发模块(C++)：实现轻量级线程池，将解析完成的 HTTP 请求分发给工作线程处理，避免频繁创建和销毁线程的系统开销。

3.协议解析模块(C++)：利用有限状态机(FSM)逐行解析 HTTP 报文，提取 GET/POST 请求中的关键信息。

4.连接管理模块(C++)：使用定时器（升序链表或小根堆实现） 管理非活动连接，自动清理超时客户端，保障系统资源不被浪费。

5.AI 推理模块(Python)：基于异步框架 FastAPI 封装大模型接口，实现对话上下文管理及流式数据处理。

在实验和测试方面，进行功能测试与性能评估：

功能完整性测试:验证从“网页输入查询”到“C++接收转发”再到“AI 回复”的完整闭环流程。

并发压力测试：利用 Webbench 或 Apache Bench 工具，测试系统在不同并发数下的 QPS 和响

应延迟。

稳定性测试：模拟长时间高负载运行，观察系统内存占用情况，验证定时器模块是否能有效防止连接泄漏。

系统功能模块图如图 1 所示。

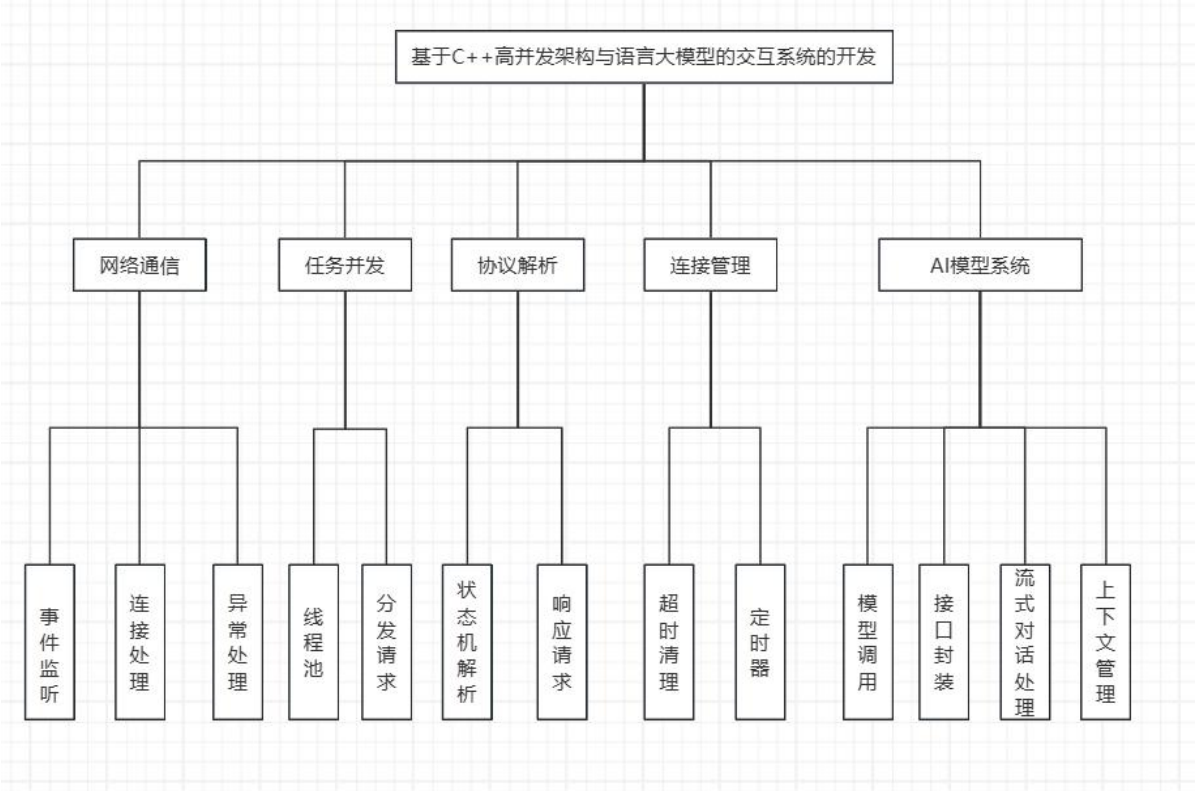

图 1 功能模块图

系统总体结构图如图 2 所示。

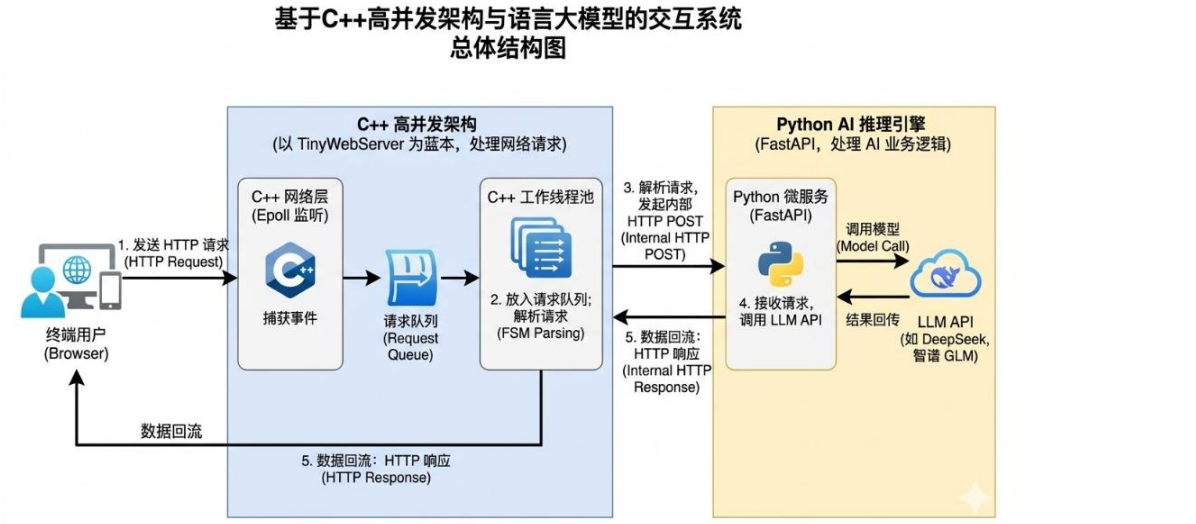

图 2 总体结构图

5、方案的可行性分析

（1）经济可行性

本项目主要依托开源框架（如 TinyWebServer、FastAPI）进行开发，在普通的云端虚拟化环境下，依然能够高效运行高并发服务器，且性能损耗在可接受范围内[9]。此外，大多数主流大模型 API 为学生和个人开发者提供了免费额度。

（2）技术可行性

C++的 Epoll 模型与多线程并发技术在工业界已有数十年的应用经验，技术方案非常成熟，甚至能直观通过三指标:吞吐量、响应时间和并发带宽来评估 Web 服务器的性能[8]。Python 的 FastAPI 框架与 LLM API 的接入文档非常完善，能够实现底层高性能与上层智能化的完美“缝合”。

（3）操作可行性

由于采用了微服务解耦设计，C++端与 Python 端可独立运行，极大降低了环境调试的复杂度。前端基于浏览器访问，无需用户安装任何客户端插件。

6、时间进程

2026 年 3 月 2 日-2026 年 3 月 15 日：确定选题，撰写开题报告，开题答辩。

2026 年 3 月 16 日-2026 年 4 月 9 日：底层架构搭建与核心技术攻克。

配置 Linux 开发环境及 Git 云端仓库存档。

完成 Socket 网络编程与 Epoll I/O 多路复用模块开发。

实现 C++线程池并整合 Reactor 高并发模型。

2026 年 4 月 10 日-2026 年 5 月 10 日：业务逻辑开发与 AI 引擎集成。

完成基于有限状态机的 HTTP 协议解析与定时器管理模块。

搭建 Python FastAPI 微服务并接入前沿大模型 API。

实现 C++与 Python 的双端联调，完成智能交互核心功能。

2026 年 5 月 11 日-2026 年 5 月 24 日：系统压力测试与性能优化。

利用 Webbench 等工具进行高并发压力测试，排查潜在的 Bug。

针对测试数据进行性能优化，确保系统高负载运行的稳定性。

2026 年 5 月 25 日-2026 年 6 月 10 日：毕业设计说明书撰写、查重与定稿。

2026 年 6 月 11 日-6 月 16 日：制作毕业设计演示 PPT，准备毕业答辩。

7、参考文献

[1]吴国栋,秦辉,胡全兴,等.大语言模型及其个性化推荐研究[J].智能系统学报,2024,19(6):1351-1365.

[2]周昆,朱余韬,陈志朋,等.YuLan-Chat:基于多阶段课程学习的大语言模型[J].计算机学报,2025,48(1):1-18.

[3]陈硕.Linux 多线程服务端编程:使用 muduo C++网络库[M].北京:电子工业出版社,2013.

[4]秦龙,武万森,刘丹,等.基于大语言模型的复杂任务自主规划处理框架[J].自动化学报,2024,50(4):862-872.

[5]W.Richard Stevens.UNIX 网络编程卷 1：套接字联网 API(第 3 版)[M].北京:人民邮电出版社,2015.

[6]游双.Linux 高性能服务器编程[M].北京:机械工业出版社,2013.

[7]尹圣雨.TCP/IP 网络编程[M].北京:人民邮电出版社,2014.

[8]Kolhar M, Alameen A. Web server performance evaluation in a virtualisation environment[J].International Journal of Advanced Computer Science and Applications, 2017,8(2).

[9]Ojesanmi O A, et al. Performance evaluation of web servers using response time and bandwidth[J].International Journal of Software Engineering & Applications,2022,9(12).

指导教师意见:	毕业设计开题答辩小组意见:
是否同意开题： ☑ 同意　　☐ 不同意 教师签字：高辉　侯亚斌 2026 年 3 月 11 日	是否同意开题： ☑ 同意　　☐ 不同意 组长签字： 2026 年 3 月 13 日